Introduction

Chapter 1

Background

1.1 Analyzing Algorithms

An algorithm is a list of specific instructions written to achieve some goal. The goal varies, and in casual conversation individuals have come to associate *algorithm* with the phrase "my algorithm" or "the algorithm," an entity with the goal to create profit for big tech companies by keeping users engaged on their apps. While those algorithms exist, they are only one type of algorithm. A cookie recipe is an algorithm, providing instructions a baker follows to yield a batch of cookies upon completion. GPS devices provide algorithms for driving from one location to another. In computer science, an algorithm is a sequence of lines of code that perform a computation, usually mathematical calculations or processing data. Because algorithms are so versatile, and there are so many different ways of accomplishing one goal, we analyze algorithms by their efficiency: how many resources they use and how much time they take to complete the task.

1.1.1 Big-O

To analyze efficiency, we have to define what we mean for an algorithm to be "efficient." For the sake of this thesis, we will focus on the time aspect of efficiency. The amount of time an algorithm takes to execute from start to finish is known as the *running time* of an algorithm. We can mathematically formalize running time with what's known as *Big-O* notation.

Big-O notation is a form of *asymptotic* analysis, or analysis that takes place as the size of the input becomes infinitely large. It is important to analyze algorithms asymptotically so that we know the algorithm is still effective in the worst case scenario. This allows us to guarantee an algorithm scales well for arbitrarily large inputs. For instance, if I am following an algorithm to bake cookies and the steps involve rolling out the dough, mixing, and measuring ingredients by hand, the algorithm works well for me to make a batch of 24 cookies. However, if I start a popular cookie franchise and suddenly I need 2,400 cookies a day, my original algorithm is not going to be the

most efficient way to make that many cookies. While it may make sense when baking cookies to handle these two cases differently, in many computer science problems data commonly scales to very large sizes, so it is helpful to know our algorithms will still be efficient in those cases.

We *estimate* running time through asymptotic analysis. In Big-O notation, we write that for some algorithm called f, $f(n) = O(g(n))$. This is read "$f(n)$ is big O of $g(n)$," where f on input n will take **at most** $g(n)$ time to run. The big O signifies the "at most" in that statement, or more formally it denotes an *upper bound* on the asymptotic running time. Running time can be classified into different categories. The most relevant categories to this thesis are constant time, polynomial time, and exponential time. Constant time, $O(1)$, can be abstractly thought of as any operation that takes the same fixed amount of time regardless of the input size. Constant time is the fastest running time an algorithm can have. Most algorithms involve more complicated operations than those that can be done in constant time, however, so our next category of time analysis is polynomial time.

Recall that a polynomial is a mathematical expression that combines terms of variables and constants using various operations (e.g. x^4+2x^3+7 or x^2+y^2). In time analysis, we simplify to the largest term in a polynomial and we disregard coefficients due to the fact that they become irrelevant as the input scales. For instance, say we have a number x^{100} versus $3x^{100}$. If $x = 1,000$, then both of these numbers are extremely large regardless of if we multiply it by 3. So in big-O notation, we would say $f(x) = 3x^{100} = x^{100}$. Similarly for multiple terms, in $x^{100} + x^5$ the larger x^{100} term will be the part that grows bigger, so when analyzing time, it overshadows the x^5 term. When we think about polynomial time, we think about how many actions must be completed per input to the algorithm. For instance, if I already have cookie dough and I need to bake the cookies, my algorithm might be: for every tablespoon of cookie dough, roll the dough into a ball and then sprinkle cinnamon on top. That is 2 actions to complete for every tablespoon of cookie dough. So, if we have n tablespoons, we could say this step takes $O(2n) = O(n)$ time (note this would be more specifically linear time, but linear time is a subset of polynomial time). For our purposes, we can consider polynomial time to be sufficiently "efficient," with the consideration that we would hope for our algorithm to be much closer to $O(n)$ time than $O(n^{100})$, as the former is much more efficient than the latter. Our last category is exponential time, which is very slow compared to our other two categories. We consider an algorithm "inefficient" if it takes exponential time. An example of exponential time in Big-O is $O(2^n)$. As n grows larger in this case, the running time increases extremely rapidly. To compare polynomial time and exponential time, imagine we have some number $n = 1000$. If we square n, we get a polynomial expression: $n^2 = 1,000,000$. That is indeed large, but not compared to 2^n, which is "a number much larger than the number of atoms in the universe" [1].

1.1.2 P vs NP

In time complexity theory, the field of mathematical analysis centered on the running time of algorithms, we give names to the aforementioned categories. Specifically, the most important complexity category for our purposes is P, which denotes the class of problems that can be solved with polynomial time algorithms. That means we can think of P as a collection of problems where every problem in the collection has an algorithm that will find a solution in polynomial time. Perhaps intuitively, we know there exist many complex problems that take longer than polynomial time to solve (such as those that take exponential time and belong to the class E). A less intuitive fact is that there also exist problems that we think do not belong in P, but we don't know how to prove it at this time nor do we believe we will know how to prove it anytime soon.

To understand what I mean by this, we need to discuss another important complexity class called NP: the class of problems solved by *nondeterministic polynomial* time algorithms. In very simplified terms, one could think of "nondeterministic" as a brute force approach: we take all possible solutions to a problem, and we have an algorithm that takes polynomial time to determine whether each solution actually solves the problem by going through them sequentially until it finds a solution that works. If the first solution happens to solve the problem, then the algorithm has been carried out once, so a polynomial number of operations have been performed. Therefore, the algorithm terminated in polynomial time. As an example, lets think about the classic 52-card game of solitaire. It is hard to know whether a starting configuration of cards will lead to a win in solitaire, but we can figure out if it is a winnable configuration by go through every single subsequent set of moves that leads to a terminating position and see if that final position is a win. If the first set of moves happens to lead to a win, we have moved a polynomial number of cards, and therefore we determined that the configuration is winnable in polynomial time [2].

It is not possible to guarantee that the first evaluated solution to an algorithm will always solve the problem, but we can imagine that if there were n possible solutions to an algorithm and we happened to have n computers that can all run the algorithm on a distinct solution, one of those computers would run the lucky input and solve the problem in polynomial time (if a valid solution exists). We could not determine which computer it would be, but we would know it exists. In real life, we do not have n computers for every problem we wish to solve with n possible solutions. However, if we were somehow given the solution output by that lucky computer, we could *verify* that it is correct by running the lucky computer's input on the real computer we do have in polynomial time. All of this is to say that NP is the class of problems that have algorithms for which we can verify a given solution is correct in polynomial time. We cannot necessarily find that solution in polynomial time, but if it was given to us we can prove it solves the problem in polynomial time.

Keep in mind that problems we know are in P are also in NP, because we can verify that their solutions (which we can find in polynomial time) are correct in polynomial

time. We will denote the problems that we cannot prove are or are not in P as $NP \setminus P$, which means the set difference of NP and P, or all problems in NP that are not known to be in P. Generally, we consider the class of problems in $NP \setminus P$ to be very difficult to solve, as it means no one has ever discovered an algorithm that can find a correct solution in polynomial time. The hardest NP problems are known as *NP-complete* problems, where "complete" means the problem can be generalized to take the form of every other problem in NP. So, if a polynomial time algorithm was discovered for an NP-complete problem, it would mean that every NP problem could be solved in polynomial time, and therefore all of those problems would actually just be in P [1].

1.2 Graph Theory

In discrete mathematics, a graph is a collection of nodes called *vertices* connected by *edges* (denoted $G = (V, E)$ where G is the graph, and it is composed of a set of vertices V and a set of edges E). Mathematicians like graphs because their structure makes them very effective for modeling a variety of real-world situations such as social networks (where vertices are individuals, and edges connect people to their acquaintances) or even the spread of disease (where vertices are people and edges represent an infection from one person to another).

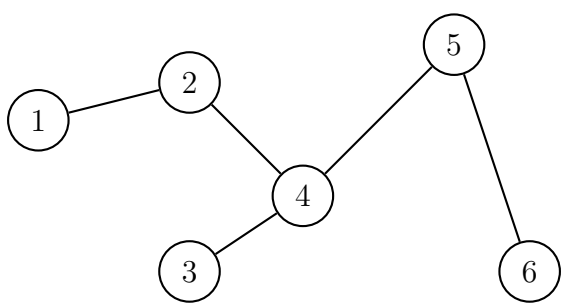


Figure 1.1: Simple graph with 6 vertices and 5 edges

Graphs can famously be used to model scheduling problems by adjusting their structure slightly. A basic graph problem has vertices and edges, but we can add factors such as *direction* going from one vertex to another, or some number of *colors* with which we want to color vertices such that no two adjacent vertices (vertices connected by an edge) are the same color. The latter example is actually one of the most studied NP-complete problems: is it possible to color a graph G using k colors? It is also the structure of graph under which scheduling has been extensively studied, where vertices represent events, edges encode overlapping time conflicts between events, and the number of colors is the number of available time slots into which those events can be scheduled [3].

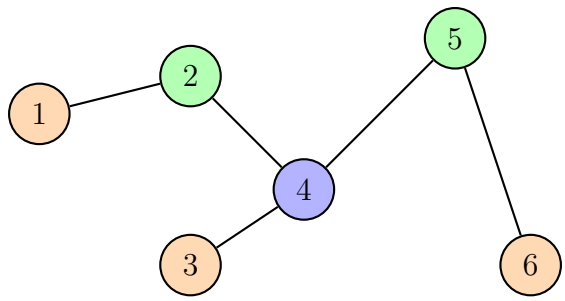


Figure 1.2: Graph from Figure 1.1 colored with 3 colors

Another way to construct graphs is to add *weights* to edges. A weight in this context is some value derived based on the constraints of the problem, and this form of graph often lends itself to optimization. As in, the weighted graph can be used to model problems where we want to maximize or minimize the weights. For instance, we can construct a graph where each vertex represents a city and each edge represents a highway from one city to another. If we were planning a roadtrip, we could add a weight to each edge to signify the travel time of that highway from one city to another. Then, if we were planning a road trip to all the cities on the graph, we could find the fastest route by finding the path with the smallest weight.

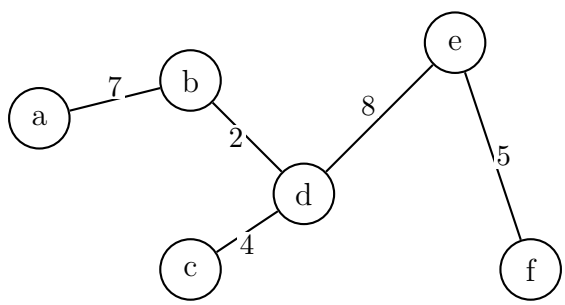


Figure 1.3: Graph from Figure 1.1 with weights

1.3 Mechanism Design

Mechanism design has traditionally been studied by economists and game theorists, though in recent years computer scientists have increasingly joined the field because mechanisms are often just algorithms. Mechanism design is the construction of systems that "fairly" (under various definitions of the word) address scenarios involving a host of participants with adverse or overlapping goals. Some examples of problems that fall under this classification are as follows: designing a voting system that successfully eliminates strategizing by any party [4], auctioning search engine advertisement spots in a way that promotes honest bidding [5], assigning roommates from a pool of students such that each student ends up with a roommate they prefer [6], and maximizing aid while allocating food donations to food banks [7].

One of the most standard metrics for evaluating the effectiveness of a mechanism is whether it is *strategy-proof*, or prevents participants from taking advantage of the rules of the mechanism for their own gain. Many more metrics exist for evaluating subcategories of mechanism design, but almost all mechanisms can be evaluated under strategy-proofness. There are two subcategories of mechanism design relevant to this thesis: stable matching and fair division.

1.3.1 Stable Matching

Many real world scenarios can be modelled as matching problems, such as assigning medical students to hospitals for residency [8], or the aforementioned roommates example (known as the Stable Roommates problem). The standard (and heteronormative) matching problem is known as the Stable Marriage Problem [9], in which the goal is to match an equal number of men and women such that no participant remains uncoupled. This is known as a *bipartite* matching problem, or one in which the market of participants is divided into two disjoint sides and matched from one side to the other. Each participant provides a list of preferences over the other side, where they rank those whom they would most like to be matched to. These are known as *two-sided* preferences. An essential aspect of the Stable Marriage Problem is that the matching should prevent "cheating," which occurs when a man and a woman who weren't matched to each other prefer to be together over their assigned partners and will "elope," leaving the other two participants alone. This couple would be known as a *blocking pair*, because their elopement "blocks" the matching from being complete. Herein lies the stability aspect of stable matching: we wish to find an algorithm that outputs a complete matching of couples with no blocking pairs.

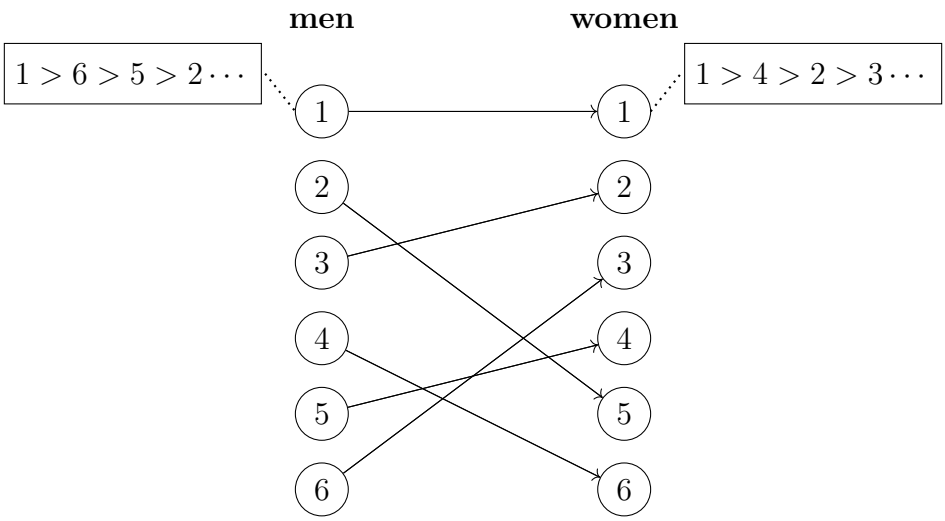


Figure 1.4: Bipartite directional graph illustrating the Stable Marriage problem

In 1962, David Gale and Lloyd Shapley published a seminal paper for the field in which they proposed the **deferred acceptance** (DA) algorithm, a solution that

guarantees a stable matching for the Stable Marriage problem in polynomial time [10]. Gale and Shapley also proved their DA algorithm could solve the College Admissions problem: matching high school student applicants to colleges. College Admissions differs slightly from Stable Marriage: it is a *many-to-one* matching in which colleges accept multiple students rather than each college matching with a single student, and it has *incomplete preferences* because students do not rank (apply to) every possible college on the list. With a slight adaptation of the DA algorithm, it can solve this version of College Admissions. Since 1962, the DA algorithm has been adapted to fit many more problems with additional factors, including the residency problem.

However, there remain subsets of problems that do not fall under the same paradigm as the DA algorithm: for instance, problems with a one-sided market. The Stable Roommates problem from above is an example of a one-sided market, in which participants are matched within one undivided pool of participants rather than between two distinct categories. Stability in a one-sided market retains the same meaning as in a bipartite setting. Over 20 years after Gale and Shapley's paper, Robert Irving published a paper with a solution to the Stables Roommates problem, in which he proved his algorithm outputs a stable matching of roommates in polynomial time if one exists [6]. Irving's algorithm, similarly to the DA algorithm in bipartite markets, provides the groundwork for the majority of research on one-sided matching problems.

1.3.2 Fair Division

The difference between two-sided matching and fair division (also known as fair allocation) is most often *one-sided* preferences. "One-sided" preferences has a different meaning than a "one-sided" market: one-sided preferences still means a bipartite market, but only one side cares what they are matched to. Because only one side cares, we no longer have the idea that "cheating" could occur; instead, we evaluate fair division under *envy*. Say we have a cake that is half chocolate and half vanilla, and we are dividing the cake between people at a party. A person who prefers vanilla but receives chocolate will be *envious* of those who receive a vanilla slice. In a fair division problem, the goal is to minimize the amount of envy between participants and maximize the amount of *welfare*, where welfare is the benefit participants yield from the goods they receive (such as feeling significantly more joy from vanilla over chocolate).

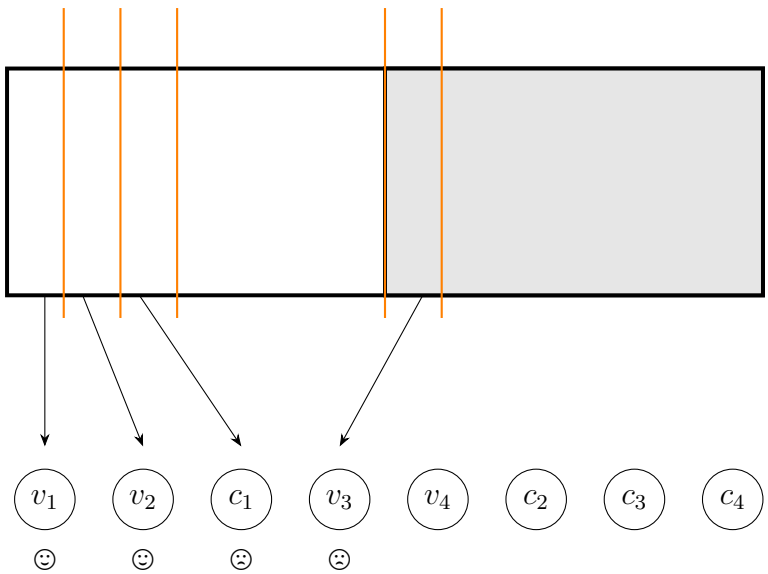


Figure 1.5: Cake cutting example where c_1 and v_3 would envy each other

The cake-cutting example is the canon example for divisible goods [11], but there is also a lot of study into the allocation of indivisible goods: items that cannot be cut up like a cake. The aforementioned food bank example, allocating donated items (such as non-perishable cans of food) to different food banks based on preferences they provide, involves indivisible goods [7].

In addition to envy and welfare, fair division will often be evaluated under Pareto-optimality. An allocation is considered *Pareto-optimal* if there is no other allocation that is strictly better for at least one participant while retaining the same welfare for the other participants [12]. So, we say allocation A *Pareto-dominates* allocation B if allocation A is better or the same as allocation B for every participant. As more factors are added to fair allocation problems—such as considering the order in which goods arrive for distribution, scaling the number of participants, and more specific definitions of welfare—envy-freeness and Pareto-optimality become more difficult to achieve.

1.4 Adding Constraints

What's known as the "vanilla" versions of these matching and division environments, which is to say the more simple or basic versions, have been studied thoroughly by now. So, to advance mechanism design research and to more accurately model real-world scenarios, scientists add *constraints* to the problems. Constraints come in many forms, and often make problems so complex they become NP-complete. In the scope of class assignment and related matching problems, constraints often come in the forms of quotas on class capacity and categories of students.

1.4.1 Quotas and Diversity

Quotas in allocation and matching problems can be implemented as upper and lower bounds on the mechanism. Upper bound quotas imply capacity constraints, such as a maximum number of open positions per hospital in the residency matching problem. Gale and Shapley's College Admissions problem, in which a college with n applicants can admit up to q students, is traditionally studied with these capacity constraints (where q is for quota). Lower bound quotas in something like the College Admissions would encode in the mechanism that schools need to accept at least m students of some type [13]. Strict quotas are one such constraint that can cause finding a stable matching in a bipartite setting to be NP-hard[14], so quotas are often studied under "soft" constraints as well: for example, allowing a quota to be a target range rather than a specific number.

This quota-based extension of the College Admissions problem more accurately reflects reality by acting as diversity constraints. The goal of diversity constraints is usually to produce a matching/allocation in which some type of participants is spread evenly or represented accurately in the result. In the actual college admissions problem, these constraints are commonly known as Affirmative Action, in which there are upper-bounded quotas on majority students to give space for minority students. There are many papers that mathematically study that particular implementation of diversity, as well as prove that it does not always benefit the minority applicants [15]. As a result, diversity constraints can take many forms—they have been studied under frameworks of lower-bound quotas [13], proportionality (evenly distributing types of students in the matching) [16], and group fairness (ensuring each distinct group of students is as happy with their matching as possible based on the happiness of each individual student in the group) [17]. In each of those papers implementing diversity constraints, the constraints make the problems NP-complete.

1.4.2 Optimization

When we work with an NP-hard or especially an NP-complete problem, we generally do not attempt to find an algorithm that gives an exact solution; the fact that the problem is NP-hard means we don't know if it is even possible to find an exact solution efficiently. However, as we have seen with quotas and diversity, many problems with relevant real-world applications are in NP. Does that mean we simply give up trying to find a solution?

Maybe we have to let go of the idea of an exact solution, but that doesn't have to stop us from trying to find a solution we deem *good enough*. "Good enough" can mean many things depending on the problem, which is the driving force behind *optimization*. Optimization problems attempt to find a near-optimal (if not optimal) solution under given constraints. Often, these will take the form of *approximation* algorithms, which will relax some of the constraints or allow a violation of the constraints up to some upper bound to guarantee a solution that is as close to the optimal solution as mathematically possible. We can clarify what we mean by "near-optimal" based on

the optimal solution: if an optimal solution has some value (such as a stable matching with no blocking pairs), we want to find a solution that maximizes/minimizes how close we get to that value (such as a matching that guarantees a maximum of one blocking pair). Another optimization approach is with *heuristic* algorithms, which leans even more into the "good enough" mentality by focusing on the empirical results. Rather than making mathematical guarantees about the optimality of the solution, a heuristic algorithm will be evaluated for its performance on actual data. Usually, despite the worst-case scenario being mathematically infeasible, the algorithm is able to output a solution when given real scenarios, and therefore we consider it to be a reasonable solution. The goal of both approximation and heuristic algorithms is to find an adequate solution quickly when the optimal solution cannot be reasonably found. There are other approaches to solving optimization problems, but these approaches are the most relevant to this thesis.

1.5 Related Work

The problem we discuss in this thesis is related to two extensively-studied problems: school choice and course allocation. School choice in mechanism design is the problem of assigning high school students to high schools such that parents are satisfied, each student has a school to attend, and no school admits more students than they can accommodate. Course allocation in mechanism design is the problem of constructing a system to assign students to classes based on which classes they prefer to take.

1.5.1 Two-sided matching

In 2003, Abdulkadiroğlu and Sonmez published a seminal paper on school choice, in which they view the problem through the bipartite lens of schools and students each with distinct preferences. This lens allows them to apply the Gale-Shapley DA algorithm as a solution as well as their own algorithm, which they called Top Trading Cycles (TTC) [18]. In Top Trading Cycles, if the matching has a cycle of students in which each student prefers the next student's school, each assignment is given to the next student until the cycle is broken.

Six years later, in 2009, Abdulkadiroğlu, Pathak, and Roth amended the 2003 paper with a case study into the New York City high school allocation mechanism, scrutinizing the DA algorithm under strategy-proofness and stability. They show empirically that there is a loss of Pareto efficiency in the algorithm when adding strategy-proofness and stability to the environment [19].

Many papers, including some outside of school choice specifically, extend one or both of Abdulkadiroğlu's papers in further research. In 2010, Biró et al. studied school choice in a college setting with upper and lower quotas, constraining the environment in a new way. They determined that stable matchings do not always exist with those quotas, and in fact determining whether a stable matching exists at all is NP-complete [13]. Three years later, Hafalir et al. analyzed the DA algorithm for school choice

under quotas for diversity, where they found that using upper quotas produced a matching that was Pareto-dominated by a matching with lower quotas for minority students [15]. There has since been much more research into the stability effects of quotas on the two-sided school choice problem, including papers that implicate stability with diverse students of multiple types [20].

1.5.2 One-sided matching

Since 2003, research also branched from two-sided school choice into one-sided matching, where only one side of the market has preferences. This was often studied under the framework of course allocation, a mechanism design problem which arose from the existing school choice literature. In 2011, Budish and Cantillon published the seminal paper on course allocation at Harvard (which cites Abdulkadiroğlu's 2003 and 2009 papers) in which they evaluate Harvard's mechanism under strategyproofness and Pareto-optimality [21]. There have been slight alterations to their framework since then, such as the effects of incomplete preferences on participant welfare.

Only in recent years have scholars started studying one-sided matching with quotas and other diversity constraints. In fact, the constraints closest to that in this paper (groups of students as well as sub-types within the groups) only seem to exist in literature published as recently as 2025. Specifically, Santhini et al. researched the empirical effects of both soft quotas [14] and group fairness [22] on the mechanism using convex optimization (also done similarly by Panda et al. [17]). There is not readily as much theory-based research on one-sided matching with constraints as two-sided matching, and those that do study it under a theoretical framework use more of a fair division analysis than stability.

As far as we can tell, the closest paper to our construction of stability in this thesis is an unpublished 2011 paper by Abdulkadiroğlu, who generalizes the two-sided and one-sided settings to one environment where the TTC algorithm can be reduced to work for both kinds of preferences. In the paper, he frames stability to be "if, whenever a student prefers a school to her assignment, that school is enrolled up to its capacity by students who have higher priority at that school" [23], where "higher priority" in the one-sided setting only considers how strongly the other students prefer that school. The paper does not consider the effect of adding diversity constraints.

Chapter 2

Formalizing the Problem

2.1 Introduction

The existing research on school choice and course allocation would seem to suggest that for our problem, which only features one-sided preferences as opposed to preferences on both sides of the market, we follow a fair division approach similar to that of Budish's one-sided matching [21]. However, given the practical application of our problem as a mechanism that is meant to reduce the Registrar's workload, we determined that constructing a notion of stability makes more sense than evaluating envy in the matching. While we care about student happiness in our solution, our formulation of the problem is primarily driven by the goal of assigning year-long HUM sections that are balanced both in size and student majors, meaning we want to minimize the desire of students to swap with each other post-assignment so as not to disrupt the balance of students. If no student wishes to swap with another student, then we can consider the assignment stable. This is the framework under which we shall formalize our model.

2.2 The Model

The basic components of our model are the two sides of our market: a set of students and a set of HUM class sections, each of which have a certain number of seats based on the class capacity. Though we are working in a bipartite setting, only one side of our market has preference over the other (i.e., students prefer certain classes, whereas classes are inanimate and therefore indifferent to students...as far as we know). Students also do not necessarily have ranked preference over every single available section; they likely equally do not want to take any sections that conflict with their other selected courses. So, preference lists will be incomplete, with those sections that students have a conflict with not being included in their ranking.

Definition 2.1. The *HUM-assignment* problem consists of a finite set of Reed College students, R, and a finite set of class sections, C. Each class section $c \in C$ has a capacity q_c representing the number of seats in the class, such that we can think of each c as a set of seats where $|c| \leq q_c$. We denote the set of all seats S, where $c \subset S$ for all $c \in C$. Students rank class sections such that each student $r \in R$ has a strict preference relation $\succ_s$ over $C \cup \{\emptyset\}$, where $\emptyset$ represents the outside option of a student such that $\emptyset \succ_s c$ for some $c \in C$ denotes that the class section is not an option for the student.

Definition 2.2. A *1-sided matching (1M)* of students to class sections is a function $\mu : R \to S \cup \emptyset$ such that

- Each student is assigned to a seat in a class section they prefer or no seat: $\mu(r) = \emptyset$ or $\mu(r) \succ_r \emptyset$ for all $r \in R$.
- No class exceeds its capacity: $|\{r \in R : \mu(r) = s_c\}| \leq q_c$ for all $s_c \in c \subset S$.

Finding a 1-sided matching would solve an instance of the HUM-assignment problem as defined above. However, we still have to introduce the last constraint of the problem: *group fairness*.

Students can be divided into three groups based on the way Reed College divides subjects: Group 1 for language, arts, and literature, Group 2 for social sciences, Group 3 for math and lab sciences [24]. To implement the HUM philosophy, which is that the class provides an opportunity for students from all knowledge backgrounds and academic interests to discuss relevant texts, we want to ensure a distribution of students that "fairly" represents each group in every HUM section. Practically, this means avoiding scenarios such as all Chem 101 students with the same lab being placed in the same section due to the overwhelming similarity in their schedules. This will come in the form of *quotas or caps* on the number of seats per section that can be assigned to each group of student, so each class will be divided into four groups of available seats: A, B, C, and free seats to allow for some flexibility.

Definition 2.3. Under the group fairness constraint, R is partitioned into three sets: $A, B, C \subset R$, where $A \cup B \cup C \subseteq R$ and $A \cap B \cap C = \emptyset$. Denote students in A as $a_1, a_2, ..., a_i$, students in B as $b_1, b_2, ..., b_j$, and students in C as $c_1, c_2, ..., c_k$.

Definition 2.4. For each class $c \in C$ with capacity q_c,

$$|c| = q_c = \sum_{t \in \{A,B,C,F\}} q_c^t$$

where $A, B, C, F \subseteq c$. Similarly to Definition 2.3, sets A, B, C correspond to the groups of students. F represents the set of free seats.

Definition 2.5. A *group-fair 1-sided matching (F1M)* builds on Definition 2.2 of a 1M by adding the quality that no class is assigned more students of each group than there are reserved and free seats for those students, maintaining that no more total seats are matched for a class than its overall capacity: $|\{t \in T : \mu(t) = s_c^T\}| \leq q_c^T + q_c^F$ and $|\{r \in R : \mu(r) = s_c\}| \leq q_c$ for all $s_c \in c \subset S$.

To make things clearer, we can represent our problem as a graph to provide us with a visualization of the problem (see Figure 2.1).

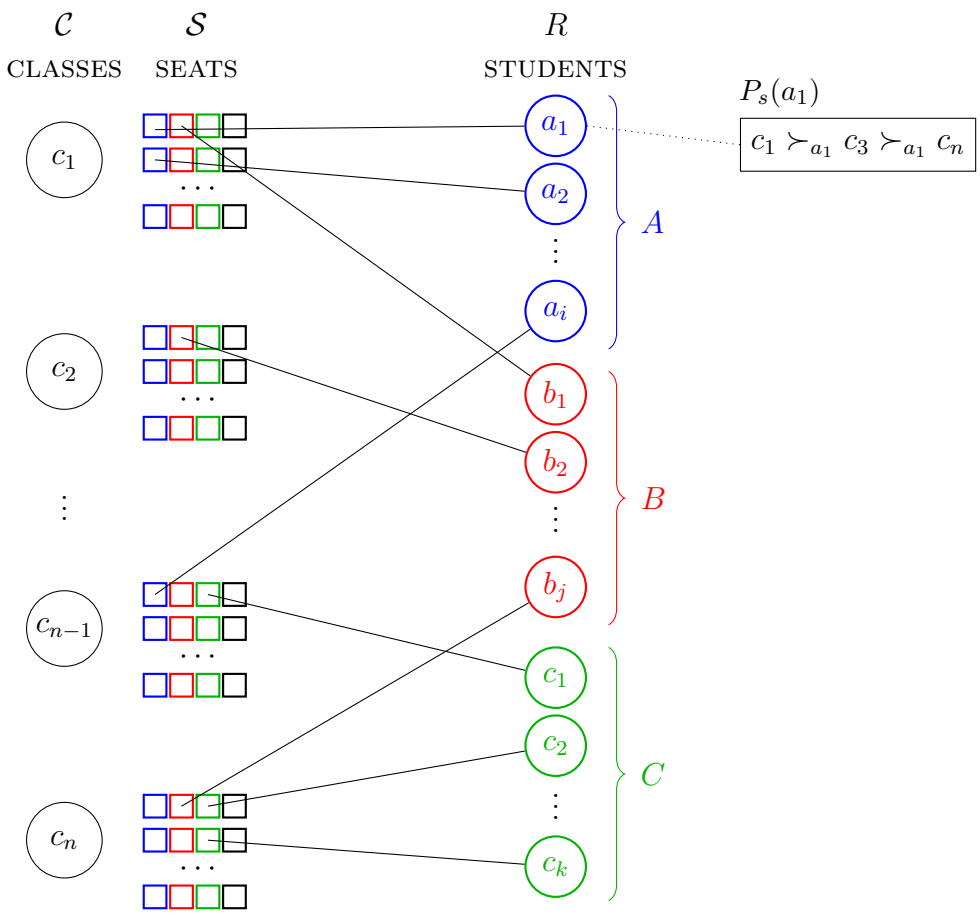


Figure 2.1: Graph illustrating an F1M.

In a graph representing the HUM-assignment problem, the vertices are comprised of students and seats that belong to specific class sections, and an edge is drawn between a student and a seat if the corresponding class is on the student's preference list. We can formally define this representation.

Definition 2.6. Let $G = (V, E)$ be a bipartite graph where:

- $V = R \cup S$
- $E = \{(r, s_c) \mid r \in R, s_c \in S, c \in C, c \succ_r \emptyset\}$

In this form, the graph only represents our problem, not a solution. For a graph to represent a solution or a matching, each student would only have one edge connecting them to a single seat in a class in a way that adheres to class capacities. So, we need a way to narrow down which edges lead to a viable solution. That means we need a way to judge edges against each other: we need to assign value to them. To

accomplish this, we can give each edge a weight that represents how conducive it is to the constraints of our problem. Recall that we want to find a solution that satisfies two goals: every student is assigned to a class they prefer, and all reserved seats are filled by students of that reserved group. We can assign weights to the edges based on these goals such that a greater weight indicates that edge embodies more of the qualities we want in our solution. Because we have two goals that are equally important, we can simply increment the weight of each edge by 1 for each goal it satisfies. We can now supplement our previous definition of a graph.

Definition 2.7. Let $G = (V, E)$ be a bipartite graph where:

- $V = S \cup C$
- $E = \{(r, s_c) \mid r \in R, s_c \in S, c \in C, c \succ_r \emptyset\}$
- $w : E \to \{1, 2\}$ is a weight function defined as

$$w(r, s_c) = \mathbf{1}\{\text{if student } r \text{ prefers class } c\} + \mathbf{1}\{s_c^T \text{ is reserved for } r \in T\}$$

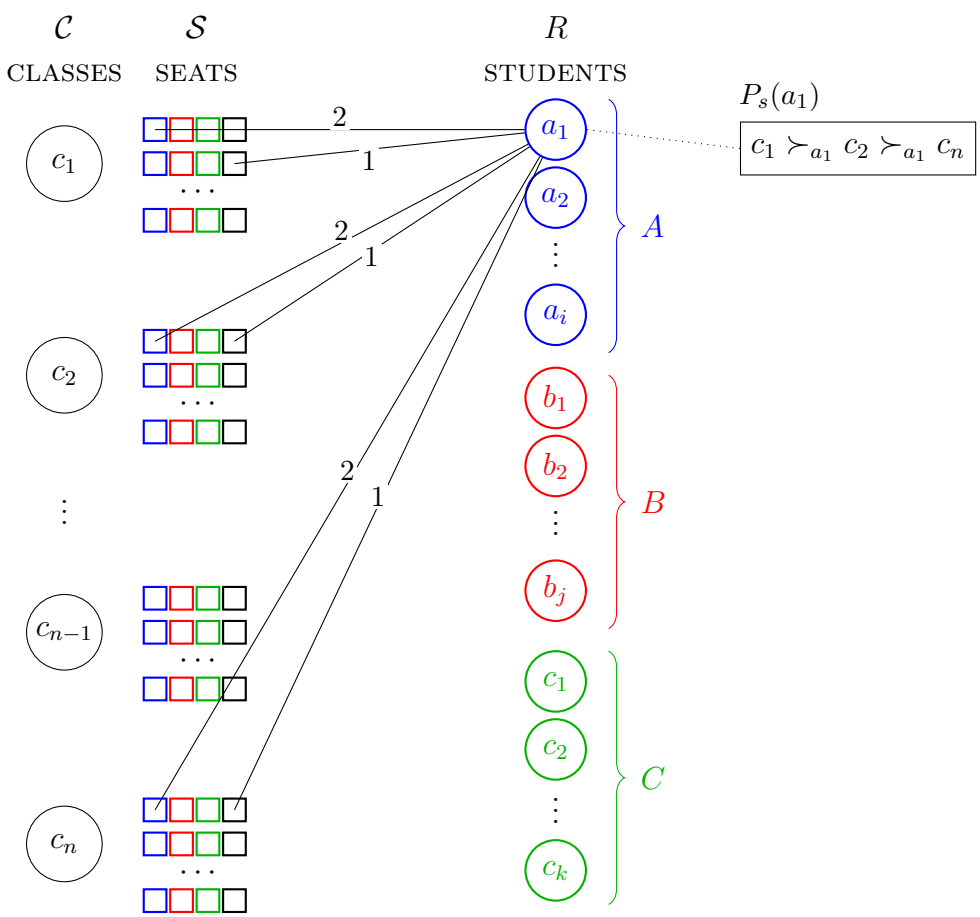


Figure 2.2: Graph illustrating weighted edges for student a_1.

Now, we have a way to compare edges to each other to allow us to find a solution if one exists. Based on the way we've set up our weights, we know that the "best" available solution (which satisfies the most constraints) will be the graph with the maximum possible weight when only one edge remains for each student. Our max-weight matching then has the properties required by our F1M (Definition 2.5), being that students of each group are prioritized assignments to classes they prefer, as well as each group being represented fairly within each class.

Definition 2.8. A matching μ is a *maximum-weight matching* if for all F1M matchings μ':

$$\sum_{r \in R} w(r, \mu(r)) \geq \sum_{r \in R} w(r, \mu'(r))$$

Observe that a maximum-weight matching would not only serve as a F1M solution, but it would also (by nature of the way we assign weights based on student preferences) ensure the solution is one in which the greatest number of students are assigned to a class on their preference list. So, viewing the problem through the lens of a weighted graph provides us with a helpful framework for satisfying the problem.

2.3 The Challenge of Stability

We've determined how to find a solution to the F1M problem from Definition 2.5, but there is one more aspect to the problem we must consider. Recall that matching is often studied under the analysis of *stability*. Notably, "stability" in our case adopts a different meaning than in a traditional stable matching problem. Because we are working with one-sided preferences, we do not consider "cheating" as a violation of stability between students and classes, but rather between students themselves. That is, we can consider a matching stable if there are no blocking pairs *between students*.

Definition 2.9. A matching μ is *stable* if there exists no *blocking pair* between students $r_1, r_2 \in R$ such that $\mu(r_2) >_{r_1} \mu(r_1)$ and $\mu(r_1) >_{r_2} \mu(r_2)$.

This notion of stability proves difficult for our problem, however. If we look back at our original definition of the HUM-assignment problem (Definition 2.1) with class capacities and incomplete preferences, we will find that in some instances it is possible there may not even exist a stable matching to find.

Theorem 2.10. *There is not always a stable matching with quotas and incomplete preferences.*

Proof. Let $C = \{c_1, c_2\}$, where there are two groups of student A and B such that for each $c \in C, q_c^A, q_c^B = 1$ (each class can have at most one student of each group). Let $R = \{a_1, b_1, a_2, b_2\}$. Let the preference lists be as follows:

$$\begin{aligned} a_1 &: \ c_1 \\ b_1 &: \ c_2 \\ a_2 &: \ c_1 \succ c_2 \\ b_2 &: \ c_2 \succ c_1 \end{aligned}$$

In this case, regardless of the order of matching or algorithm, there is no stable matching. The A students will both wish to be matched to c_1, which only has one A seat, and both B students wish to be matched to c_2 which only has one B seat. So, when the first round of matching is done, that means that there is still an open B seat for c_1 and an open A seat for c_2, which the unhappy A and B students can then switch into on their own, respectively. □

Defining stability in this way is, then, too strict. Recall that even determining whether there is a stable matching or not is NP-complete in many scenarios similar to this one [14]. So, we need to relax our constraints to allow us to explore some less restrictive notion of stability. For now, let's simplify the problem. At a basic level, before we even think about stability, we want to work in an environment where we can be sure an algorithm would respect all student preferences, such that no student is assigned to a class they did not list as an option.

To temporarily remove our notion of stability, we can alter our preferences from a ranked system to what is known as *0/1 preferences*. In 0/1 preferences, students give a class a 1 if they are okay with being matched to it and a 0 if it is not an option for them. This way, there cannot be a notion of stability because all preferred classes have the same rank. Now, for ease of notation, we will let S_{res} be the set of reserved seats across all classes. Observe that this includes all seats other than the set of free seats, or formally:

$$|S_{res}| = \sum_{\substack{c \in C \\ t \neq F}} q_c^t$$

Notably, this simplified environment still lends itself well to the same weighted graph construction from Definition 2.7, so we can continue to use a weighted graph to represent our problem.

Theorem 2.11. *A matching fills all reserved seats while only assigning students to classes they prefer if and only if it is the maximum-weight matching where* $w(\mu) = |R| + |S_{res}|$.

Proof. First we will prove the forward direction. Let μ be a matching that assigns every student to a seat they prefer while also filling reserved seats first, such that all reserved seats are filled before free seats. By construction of the weights, we can determine a lower and upper bound for $w(\mu)$.

1. **Lower bound of μ:** Each edge exists only if the student prefers that seat, so every edge in μ contributes at least 1. By our assumption of characteristics of μ, every student in μ got a seat they prefer, so summing over all students contributes at least $|R|$ to $w(\mu)$. We also assumed each reserved seat was assigned to a student in μ, which adds an extra $+1$ to the weight for the edge from each of those reserved seats. Summing over all reserved seats gives an additional contribution of $|S_{res}|$, because the rest of the seats are still 1-weight edges. Therefore, we can bound μ:

$$w(\mu) \geq |R| + |S_{res}|.$$

2. **Upper bound of μ:** From Definition 2.7, we know each edge weight is at most 2. Similarly to the lower bound, we know μ has $|R|$ edges because each student was matched to one of their preferences. However, because only the reserved seats contribute an additional weight to an edge, we know that the number of 2-weight edges $\leq |S_{res}|$. So, the maximum summed weight of all edges must be $2|S_{res}| + |R| - |S_{res}|$, representing the maximum number of 2-weight edges plus all students assigned to an overflow seat. Note that is equivalent to $|R| + |S_{res}|$. Therefore, we can bound μ:

$$w(\mu) \leq |R| + |S_{res}|.$$

So, μ is a maximum-weight graph with a lower bound of $|R| + |S_{res}|$ and an upper bound of $|R| + |S_{res}|$, so it must be that a matching that fully respects student preferences and seat reservations has a weight $w(\mu) = |R| + |S_{res}|$.

Now, we prove the backwards direction. Let μ be a maximum-weight matching where $w(\mu) = |R| + |S_{res}|$. The term $|R|$ implies each student was assigned to a class. By our construction of G such that an edge between a student and a class only exists if the student prefers that class, the fact that each student is assigned to a class must mean it is on their preference list. Additionally, the $|S_{res}|$ term implies every reserved seat is assigned, because by our weight function each edge only has additional weight $+1$ if the edge is between a student of the same group as the seat. So, a maximum-weight matching where $w(\mu) = |R| + |S_{res}|$ is a matching that fills all reserved seats while only assigning students to classes they prefer. □

Theorem 2.11 shows us that we can identify a way to assign each student to a class they prefer while respecting group fairness constraints if such a matching exists. From here, we can begin to reintroduce stability. While it may not be feasible to find stability between every student, perhaps we can start by comparing just students of the same group. We are introducing a new notion of stability here, where we will consider a matching stable if it exhibits *intra-group stability*. This means that for a student of any group, that student does not form a blocking pair with another student *of the same group*.

Definition 2.12. A matching μ is *intra-group stable* if, for any two students t_1 and t_2 for all groups $T \subset R$, $\mu(t_1) >_{t_1} \mu(t_2)$ or $\mu(t_2) >_{t_2} \mu(t_1)$ such that there exist no intra-group blocking pairs.

With our new goal of intra-group stability, we can now add to our understanding of a viable solution to our HUM-assignment problem.

Definition 2.13. An *intra-group stable, group-fair 1-sided matching (ISF1M)* is a function $\mu : R \to S$ such that

- Each student is assigned to a class they prefer:
  $\mu(r) \succ_r \emptyset$ for all $r \in R$
- No class exceeds its capacity: $|\{r \in R : \mu(r) = s_c\}| \leq q_c$ for all $s_c \in c \subset S$
- No class exceeds its capacity for each student group:
  $|\{t \in T : \mu(t) = s_c^T\}| \leq q_c^T + q_c^F$
- There exist no intra-group blocking pairs:
  for all $t_1, t_2 \in T \subset R$, $\mu(t_1) >_{t_1} \mu(t_2)$ or $\mu(t_2) >_{t_2} \mu(t_1)$

For now, our goal will be to develop an algorithm that finds an ISF1M to solve our problem. We will classify our previous definition of stability (Definition 2.9) as *inter-group stability* (blocking pairs involving students of different groups) to draw a distinction between the two notions. For now, we will consider an ISF1M to be a successful solution to the HUM-assignment problem even though it is not inter-group stable. Though we would ideally guarantee stability between all students, looking at the problem through the lens of intra-group stability allows us to give some theoretical guarantee about minimizing the amount of swapping that will occur in an assignment while still remaining computationally feasible, as opposed to turning to less-reliable heuristic solutions in an attempt to solve the broader conception of the problem.

Chapter 3

Developing an Algorithm

3.1 First Come, First Served

As it currently stands, the sign-up process for HUM 110 classes takes place during freshman class registration. The registration software, SOLAR, opens to incoming freshmen at a certain time, and at that time (or later) all freshmen log on to the platform and choose their own academic year schedule in whatever order they desire. This includes their HUM 110 class (or, as is the driving force behind this project, classes plural). Though this process may not inherently seem like an algorithm, it can actually be modelled as the algorithm commonly known as *First Come First Served* (FCFS).

FCFS takes some ordering of students (in this case, those who sign up for HUM 110 the fastest to slowest after registration opens), and at their point of selection each student chooses their top available choice until every student has chosen a HUM class. In this case, "available" means classes that have not yet reached capacity. We can write out FCFS formally as follows, where P_s represents the ordered preference list for a student s (corresponding to their preference relation defined in Definition 2.1).

Algorithm 1 First Come First Served

```
for each student s in S do                          ▷ where S is an ordered set
    for each class c in P_s do
        if # students assigned to c < q_c then      ▷ class is not at capacity
            μ(s) ← c
            break                                   ▷ move to next student
        end if
    end for
end for
return matching μ
```

There are various benefits to a FCFS algorithm, one of the most significant for

our purposes being that the given matching will be *inter-group stable*. Because each student chooses their most preferred option when it is their turn to select, there can be no two pairs of students that wish to switch. A proof using this logic can be found in section 2 of this chapter. Additionally, in practice for FCFS, many students get their top choice of class or near to it for HUM 110.

However, FCFS has other issues. For instance, it is possible some students will not get to choose a class they want at all if all of the ones they prefer are at capacity. Self-selection also allows for inconsistent class sizes and it does not consider student groups, thus permitting an inconsistent distribution of students. Our goal in this section is to develop an algorithm that improves upon FCFS with group fairness.

3.2 A Greedy Approach

We introduce group fairness into our algorithm with what we call a *greedy* algorithm: one that attempts to maximize student happiness while following the constraints set by the group fairness. We call it *greedy* because for each student, we still assign them their best available choice at the time of evaluation. However, in addition to the class capacity constraint, "available" also considers caps on how many of each group of students can be in each class. So, a class that is not yet at capacity is still unavailable to a student of group T if it has already reached its maximum number of group T students. Essentially, GREEDY implements FCFS under the additional group fairness restriction.

Algorithm 2 Greedy Algorithm

1: **for** each student s_T in S **do** ▷ where T is the student's group
2: **for** each class c in P_s **do**
3: **if** # students assigned to $c < q_c$ and # T students assigned to $c < q_c^T$ **then**
4: $\mu(s) \leftarrow c$
5: **break** ▷ move to next student
6: **end if**
7: **end for**
8: append s to X
9: **end for**
10: **for** each student s_T in X **do** ▷ randomly assign unmatched students
11: choose random available c from C
12: $\mu(s) \leftarrow c$
13: **end for**
14: **return** matching μ

Let's compare GREEDY to FCFS. The primary differences are in line 3, where we add the group cap constraint, and the second part of the algorithm starting on line 10 assigns students randomly instead of leaving them unmatched. We can start

by showing that in a vanilla environment, in which we do not factor in student groups, both algorithms output a stable matching. Note that in a vanilla environment, GREEDY is equivalent to FCFS because unmatched versus random assignments are regarded with the same preference rank by students.

Theorem 3.1. GREEDY *outputs a stable matching in a vanilla environment.*

Proof. Let μ denote the outputted matching over an ordered set of students, S. Let $s_1, s_2 \in S$, where s_1 comes before s_2 in the order. Assume for the sake of a contradiction that s_1 and s_2 form a blocking pair, such that $\mu(s_2) >_{s_1} \mu(s_1)$ and $\mu(s_1) >_{s_2} \mu(s_2)$.

By lines 3 and 11 of GREEDY, we know $\mu(s_1)$ was either s_1's top available choice or a random assignment. So, if $\mu(s_2) >_{s_1} \mu(s_1)$, then $\mu(s_2)$ must not be available when s_1 is matched, which means s_2 was matched first. However, this is a contradiction of our assumption that s_1 comes before s_2 in the input to the algorithm. Thus, $\mu(s_2)$ is available when s_1 is matched, so it must be the case that $\mu(s_1) >_{s_1} \mu(s_2)$. Therefore, s_1 and s_2 do not form a blocking pair, and we can conclude that μ is stable. □

Now, we can formally evaluate what happens when we do consider groups. As we discussed in Theorem 2.10, we know an inter-group stable matching may not exist at all when we introduce caps on groups. This already poses a problem for GREEDY, as it means it may not always output an inter-group stable matching because one might not exist. However, even assuming there does exist an inter-group stable matching, GREEDY is still not guaranteed to find it.

Theorem 3.2. GREEDY *does not always output a stable matching with group cap.*

Proof. Let μ denote GREEDY's outputted matching over a set of students, S. We will prove by counter example that μ is not stable.

Let $S = A \cup B$ where there are two groups of students, group A and group B. Let $C = \{c_1, c_2, c_3 \mid q^a_{c_i} = 1, q^b_{c_i} = 1\}$, such that each $c \in C$ can have at most one student from each group. Let $a_1, b_1, a_2, b_2 \in S$ where students $a_1, a_2 \in A$ and $b_1, b_2 \in B$. Let the preference lists of the students be as follows:

$$\begin{aligned} a_1 &: \quad c_1 \succ c_3 \\ b_1 &: \quad c_2 \succ c_3 \\ a_2 &: \quad c_1 \succ c_2 \\ b_2 &: \quad c_2 \succ c_1 \end{aligned}$$

Observe that in μ, $\mu(a_1) = c_1$ and $\mu(b_1) = c_2$, because they are the first two students to be matched and their first choices are different, so they receive their top choices. Next, $\mu(a_2) = c_2$ because c_1 already has a student in group A so c_1 is no longer available when matching a_2, and similarly $\mu(b_2) = c_1$ because c_2 already has a student in group B. Notice that $\mu(b_2) >_{a_2} \mu(a_2)$ and $\mu(a_2) >_{b_2} \mu(b_2)$, forming a blocking pair in which a_2 and b_2 are incentivized to switch classes with each other. So, it must be

the case that μ is not a stable matching and therefore GREEDY cannot guarantee a stable matching under group cap constraints. □

The issue with GREEDY is that we are still working in our too-constrained environment from the start of chapter 2. We need to come up with an algorithm that allows us to work with our conception of intra-group stability from definition 2.12.

3.3 Weighted Trading

Let's return to our conception of the HUM-assignment as modeled by a graph. We know by Theorem 2.11 we can use a maximum-weight matching algorithm to find a matching that is group fair and assigns every student to a seat they prefer. Notably, this is only one requirement away from being an ISP1M solution as per definition 2.13. We just need to reintroduce intra-group stability to our max-weight matching. To do that, we would need to be sure that we remove all blocking pairs between students of the same group while preserving the weight of the max-weight matching, as the weight is how we determined that the matching is full and group fair. Here we define a new algorithm called the *Weighted-Trading* (WT) algorithm.

Algorithm 3 Weighted-Trading Algorithm *WT*

1: $w(\mu) \leftarrow 0$
2: **while** $w(\mu) \neq |S| + |C_r|$ **do**
3: $\quad \mu \leftarrow$ MAX-WEIGHT(S, C)
4: $\quad$ **if** $w(\mu) \neq |S| + |C_r|$ **then**
5: $\quad\quad$ reduce $|C_r|$
6: $\quad$ **end if**
7: **end while**
8: **for** each $T \subset S$ **do**
9: $\quad$ **while** $\exists$ blocking pair t_i, t_j **do**
10: $\quad\quad \mu(t_i) \leftarrow \mu(t_j)$, $\mu(t_j) \leftarrow \mu(t_i)$
11: $\quad$ **end while**
12: **end for**
13: **return** matching μ

We can break the WT algorithm into two key stages: lines 1 to 7 and lines 8 to 13.

The first stage is the *weighted* part of the algorithm: it repeatedly runs the max-weight algorithm until it finds a satisfying assignment. If the weight of the first max-weight run is not $|S| + |Cr|$ (as per Theorem 2.11) we relax our seat reservations to allow for more students of various groups to fill the class. This involves decrementing the number of reserved seats per group by one and then re-running the max-weight algorithm. If the seat reservations are relaxed to the point that there are no more reserved seats, it means there is no group-fair solution and we simply output the max-weight matching for students without considering their groups.

The latter stage is the *trading* part of the algorithm, which handles our intra-group stability requirement. We check for all existing blocking pairs between students of the same group, and we swap or trade their assignments with each other. Notably, we have to continually check each pair in a group of students until no more blocking pairs remain, as checking each student once does not account for new blocking pairs that may arise from swapping the assignments of two students. At the end of the algorithm, we have a group fair, intra-group stable matching (or an ISP1M) where each student is assigned a class they prefer if such a matching exists.

We can assert that the outputted matching after trading is still full and group fair by proving that the weight of the graph is still the maximum weight found in stage 1 of WT.

Theorem 3.3. *For a matching μ output by the* WT *algorithm, $w(\mu) = |S| + |C_r|$.*

Proof. At line 7 of Algorithm **??**, $w(\mu) = |S| + |C_r|$ because it was just output by the max-weight algorithm, assuming such a matching exists. So, we have to evaluate the for loop that compares each student of some group to each other student of that same group. In the for loop, the students swap seats if they both prefer each other's match in μ. This means that each student still prefers their new match, so the weight of the edge retains weight at least 1 from that student. Then, if either student was in a seat reserved for that group, we replaced them with a student of the same group, so the edge retains the additional weight of the reserved seat. Therefore, swapping any two students of the same group does not change the weight of μ. □

While it would be nice to guarantee inter-group stability, recall that the only known algorithms for assignment problems that consider inter-group stability and group fairness are NP-complete [14]. With the WT algorithm, we can guarantee that if an ISP1M exists, we can find it in an efficient amount of time.

Theorem 3.4. *The* WT *algorithm will output a stable matching in polynomial time.*

Proof. Let μ be the matching output by MAX-WEIGHT at line 7 of Algorithm **??**. Observe lines 8 and 9 of Algorithm **??**, which outline that within each student group, we must swap the assignments of blocking pairs until no more remain. We know this loop will terminate because of our assumption that preference lists are finite, so students will eventually reach their highest possible preference and no longer be able to form blocking pairs. The maximum length of a preference list is equivalent to the number of classes, $|C|$. So, the maximum number of times the while loop can execute per for loop is as follows:

$$|C| \times |S|$$

because at most we can traverse each student's entire preference list once. If we terminate this loop for each group of student $T \subset S$, then we must have intra-group stability because no more blocking pairs remain in each group.

We can analyze the runtime of this algorithm. Since the while loop executes at most $(|C| < |S|) \times |S|$ times, we can bound the execution of the while loop to $O(S^2)$.

Additionally, we run the while loop for each group of student. There can be at most $|S|$ groups of students, such that each student is in a distinct group, so that gives us $O(S^3)$. Therefore, the trading stage of the algorithm will run in at most cubic time. Note that before that, we run the Max-Weight algorithm at least once, which runs in $O(V^3)$ time where V is the set of vertices [25]. Our vertices are all the students and seats, so the runtime for the weighted stage is $\mathrm{O}((q_c \cdot C + |S|)^3)$. So, our total runtime for the algorithm is $O((q_c \cdot C + |S|)^3 + S^3) = \mathrm{O}((q_c \cdot C + |S|)^3)$. Therefore, the WT algorithm outputs an intra-group stable matching in cubic time. □

We now have an algorithm that theoretically gives us a viable solution to the HUM-assignment problem. The next step is to evaluate it against FCFS and GREEDY to determine whether it performs well in practice.

Chapter 4

Experiments

4.1 Data

In collaboration with the Reed College Registrar, we obtained three years of anonymized student and class schedule data for the purpose of evaluating our algorithms. The student data compiles the fall and spring class schedules chosen by every student registered as a freshman for that year, de-identified by representing each student with a randomly generated ID number. We use this information to ensure students are not matched to a section with a time conflict. The data also includes the students' top prospective major from their application to Reed, which we use for sorting students into groups. While the prospective major may not accurately reflect each student's chosen major by the time they are in class, it provides a good baseline for what discipline a student is interested in, which is sufficient for our group fairness purposes. The Registrar also provided us with the master class schedules for each year corresponding to the student data, so we can match students to the actual sections that were available for their year.

4.2 Implementation

To perform experiments, we implemented each algorithm in Python. We chose Python for the fact that it is a popular higher-level programming language, which means it will be easier for others to read and understand should this algorithm actually be implemented after this thesis is complete. It is also an object-oriented language, which means we could represent the components of our model as classes with properties, such as a `Student` class, which holds each student's provided data, and a `Class` class, which can hold information such as capacity, enrolled students, and meeting times. The object-oriented approach helps the implementation more intuitively represent our actual scenario. Additionally, Python has a very accessible csv library that allows us to parse our data into these objects. While Python may not be as fast as other languages with some similar features, we decided that because the WT algorithm

performs in polynomial time (see Theorem 3.4) and Reed has relatively small class sizes, the impact on the performance of the algorithms was less essential than the readability of the code.

4.2.1 Synthesizing Preferences

While we did have access to the student schedule and major data, we were still missing a key component of a matching problem: rankings. Because there was no feasible way of acquiring each student's actual preferences from class registration years ago, we needed to synthesize student rankings over the HUM sections to effectively simulate our stable assignment problem.

According to conversations with The Registrar throughout this process, Reed College students are generally less inclined to wake up early and take longer classes—which means that 50-minute classes held later than 10am tend to be the most popular choices. To simulate this preference distribution, we used an implementation of the Mallows Ranking Model [26] from the Python library *PrefSampling* [27]. This model is a popular way of generating ranking data for simulating elections, and is one of the more effective methods of doing so [28]. The Mallows model takes a "central vote" representing the most popular real-life choices and deviates the outputted rankings from this according to some dispersion coefficient $\phi = [0, 1]$. A ϕ closer to 1 means rankings that differ significantly from the central vote are more likely to be generated. We found a good ϕ value for our purposes was 0.4, which allows for some deviation but overall keeps the rankings more central.

In `synthesize_preferences`, we iterate through each student's schedule and find all of the HUM sections that do not conflict with their other chosen classes (including the HUM they actually chose that year). Then, we turn those available sections into a central vote by weighting them by their time and class period length, where later and 50 minutes is weighted higher. We run that vote through Mallows to generate the student's preferences, and then we truncate the list to some length to simulate other unknown time conflicts, as well as the fact that students may not have preference over every individual section. These are the preferences used when running each algorithm.

4.2.2 Notes on the Algorithms

FCFS and GREEDY both involve an ordering of students; in real-life, this is in the form of whichever students are fastest to sign up for HUM. In our implementation, we randomly shuffle the list of students and then iterate through to represent this ordering.

For the WT algorithm, we represent the graph of students to seats as an adjacency matrix. The rows represent students, and the columns represent seats in each section, so the adjacency matrix is of size $|S| \times |T|$. At each index we calculate the weight of the edge between the student and that seat according to the algorithm. For instance,

in Figure 4.1, student 0 has weight 0 for seat 0, so we place a 0 at 0,0 in the matrix. A 0 means there is no edge between that student and seat. Similarly, student 0 has weight 2 for seat 1, so we have a 2 at 0,1.

$$\begin{bmatrix} 0 & 2 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Figure 4.1: Adjacency matrix representing 3 students and 3 seats.

The adjacency matrix we generate is the input to the Max-Weight algorithm. In Section 3.3, we discuss the Hungarian algorithm as a specific Max-Weight algorithm we could use. In practice, we actually use an algorithm called *LAPJVsp*, implemented in the SciPy library [29]. LAPJVsp also performs in $O(n^3)$ time theoretically, but in practice is usually much faster [30], which is helpful when repeatedly running trials of the algorithm in our experiments.

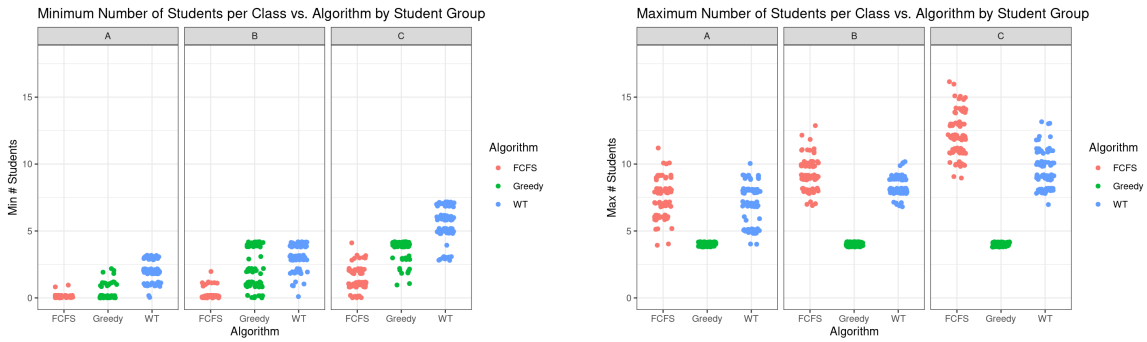
The matching returned for all algorithms is a Python dictionary with student IDs as the keys, and the ID of the class they were assigned to as values.

4.3 Results

Because of the randomness in our preference synthesis, being that Mallows does not necessarily output the same preferences each time, as well as the randomness involved in the ordering of FCFS and GREEDY, we ran many trials to accurately evaluate the algorithms' performances. The length of the preference lists was another variable that could affect whether students got a class they preferred, so we vary the trials by preference list length as well.

We ran 10 trials each on preference lists of lengths at most 5, 10, and full (not truncated) for all three years of data. Performing these trials for each algorithm gives 90 trials per algorithm.

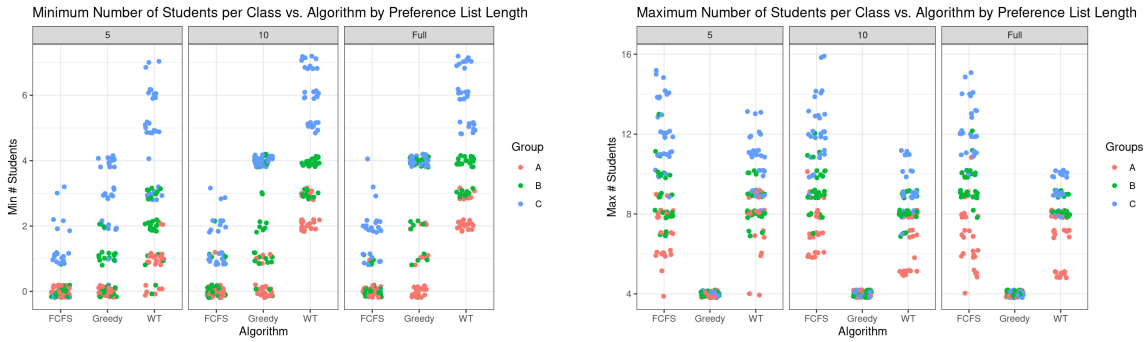
Recall our original goal for this project: assign students to classes under group fairness constraints. So, we evaluate how well each algorithm accomplishes this task. We can measure group fairness by looking at minimums and maximums: for each trial, what is the minimum number of students of each type in each section and what is the maximum? These metrics will indicate to us how well they are distributed.



(a) (b)

Figure 4.2: Comparison of minimum and maximum number of students of each group over classes in all trials.

Figure 4.2 illustrates how students are distributed within groups. Notably, in each sampled year, the set of students was approximately composed of 17% group A, 28% group B, and 53% group C, with the rest of the students undeclared. This distribution in students is reflected in the minimums and maximums. For instance, in 4.2b, the greatest difference can be seen in the plot for group C students: FCFS and GREEDY did not proportionally assign students of group C as successfully as WT in comparison to the other groups. It is also notable that in 4.2a, FCFS and GREEDY frequently assigned students such that many classes had 0 students of some group, whereas in nearly all of the WT trials there was some representation of each group in every class, thus accomplishing the group fairness goal much more successfully.



(a) (b)

Figure 4.3: Comparison of minimum and maximum number of students of each group in trials with different maximum length preference lists.

Furthermore, we can explore how the preference list truncation affects the distribution of each group. Observe in Figure 4.3a, we can see that the few trials where

the WT algorithm had classes with 0 group A or B students occurred with length 5 preference lists. When the lists were truncated at a longer length, WT always put at least 2 students of each group in each class. Additionally, both plots in Figure 4.3 demonstrate that there is not a meaningful difference between preference lists of length at most 10 and lists that have not been truncated at all.

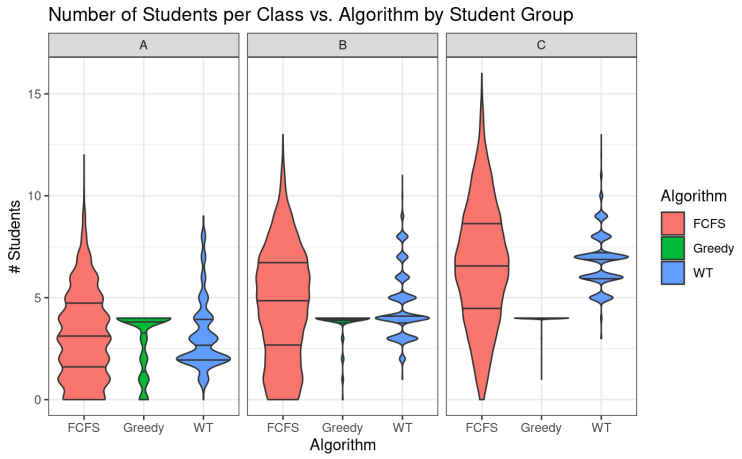


Figure 4.4: Comparison between groups of number of students assigned to each class over all trials.

We also observe the overall visualization of the distribution of groups amidst classes for all trials in Figure 4.4, which shows that WT creates a much less variable distribution of students, where most of the classes have a difference of 2 between each group. Compared to FCFS, where the number of students ranges much more (for instance, from 0-18 rather than 3-12 for group C), we can conclude better group fairness from the WT algorithm.

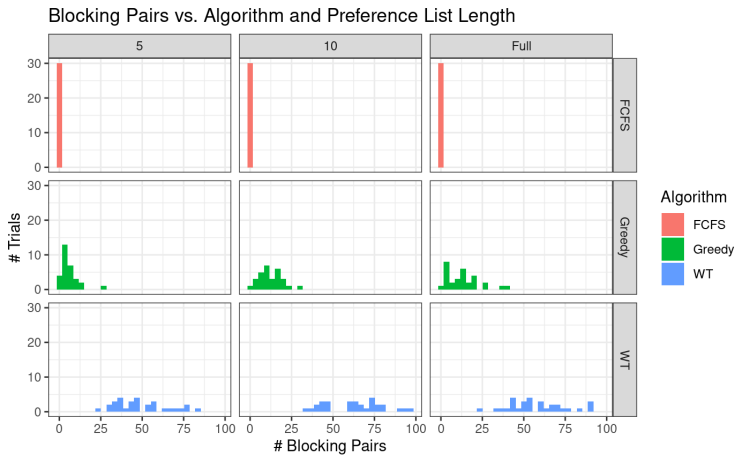


Figure 4.5: Inter-group blocking pairs present in each trial, compared between maximum preference list length and algorithm.

We also are interested in our original conception of stability, which means we want to check how many inter-group blocking pairs are present in the final matching output by each algorithm. Figure 4.5 illustrates that there are 0 blocking pairs in any matching output by FCFS, which is to be expected by the results of Theorem 3.1. We can also observe that GREEDY and WT both tend to have blocking pairs, with WT consistently having the most, which is also consistent with Theorem 3.2 and our problem framework for WT in Definition 2.13.

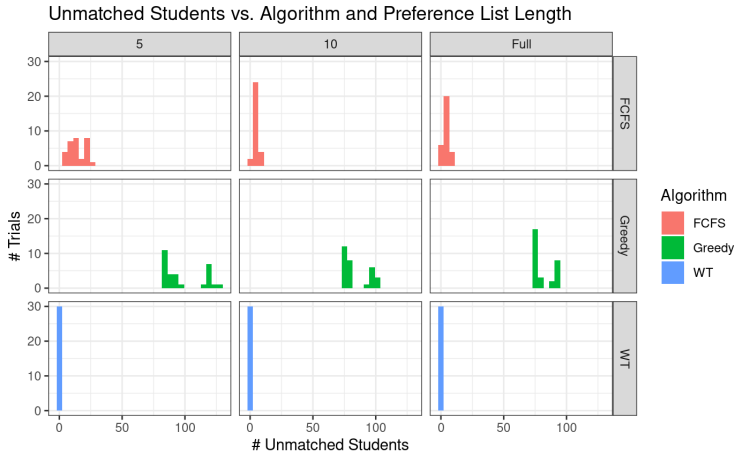


Figure 4.6: Number of unmatched students in each trial, compared between maximum preference list length and algorithm.

Relatedly, we check how many students remain unmatched in the outputted matching in each trial. While WT may produce the most blocking pairs as seen in Figure 4.5, Figure 4.6 shows that it also successfully matches every student to a year-long HUM

section in all trials. We can also observe that GREEDY often leaves between 50-100 students unmatched, and FCFS always has unmatched students with preference list length less than 10.

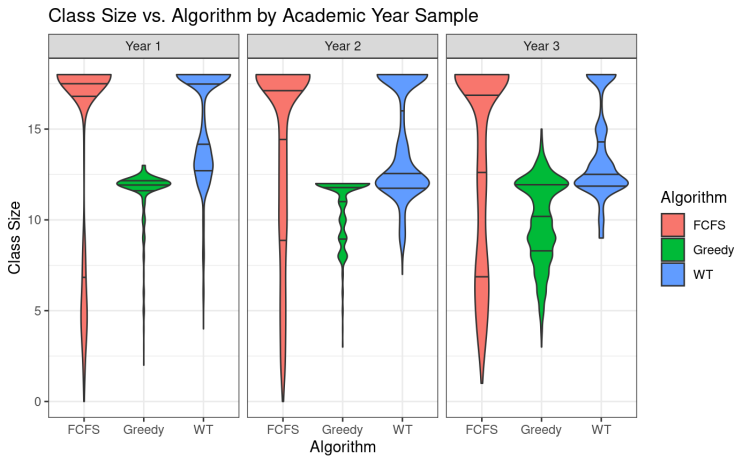


Figure 4.7: Comparison between sample years of number of students assigned to each class over all trials.

Finally, though it was not part of the design of the algorithms, we are interested in how well the sections are balanced regardless of student groups. Generally, we would hope all the sections are similarly sized. Figure 4.7 illustrates the distribution of students in each sample year. The most successfully balanced year is Year 3, in which the smallest class in the WT trials is size 9 and the largest class is size 18, giving a difference of 9. Additionally, in both Year 2 and 3 the majority of classes were size 12 or 13. While there is some range between class sizes in the WT algorithm, the figure demonstrates that it is much smaller than the range in FCFS and GREEDY.

As a whole, the WT algorithm outperformed FCFS and GREEDY on every metric we analyzed other than blocking pairs. Due to our re-framing of the problem under intra-group stability from Definition 2.12, we accept this result as successful.